2024-11-14-NetAppls

Generated by Scribe.AI

December 31, 2024

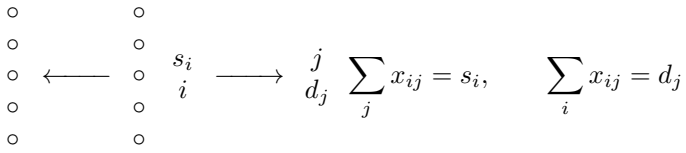
Slide 1

Content

Variants of Network Flows

Inequality constraints (Transportation Problem) [CJ] p.320

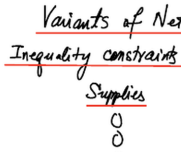
Supplies Demands



Description

Slide 1 of 21 in a Linear Programming course introduces variants of network flow problems, focusing on the transportation problem. The title "Variants of Network Flows" is underlined. The slide highlights that the transportation problem is characterized by inequality constraints, referencing a textbook ([CJ] p.320). A simple network flow diagram is presented, showing a source node i with supplies s_i and a destination node j with demands d_j. The flow between nodes i and j is denoted by x_{ij}. Multiple arcs are shown to represent the possibility of multiple flows between the source and destination. The circles represent nodes, and the arrows represent the flow direction. The key constraints of the transportation problem are given: $\sum_j x_{ij} = s_i$ (total flow out of node i equals supply s_i) and $\sum_i x_{ij} = d_j$ (total flow into node j equals demand d_j). The words "Supplies" and "Demands" are underlined. The context of the course is crucial for understanding the different types of network flow problems and their applications in linear programming.

Figures



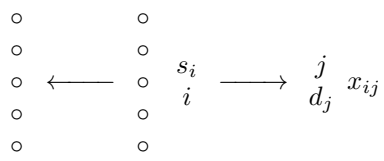
(a) Handwritten notes on variants of network flows, specifically the transportation problem.

Slide 2

Content

Variants of Network Flows
Inequality constraints (Transportation Problem)

Supplies Demands

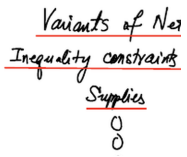


$$\sum_j x_{ij} \leq s_i, \qquad \sum_i x_{ij} \geq d_j$$

Description

Slide 2 of 21 in a Linear Programming course introduces variants of network flow problems, focusing on the transportation problem with inequality constraints. The title "Variants of Network Flows" is underlined. The slide highlights that the transportation problem involves inequality constraints. A simple network flow diagram is presented, showing a source node i with supplies s_i and a destination node j with demands d_j. The flow between nodes i and j is denoted by x_{ij}. Multiple arcs are shown to represent the possibility of multiple flows between the source and destination. The circles represent nodes, and the arrows represent the flow direction. The key constraints of the transportation problem are given as inequalities: $\sum_j x_{ij} \leq s_i$ (total flow out of node i is less than or equal to supply s_i) and $\sum_i x_{ij} \geq d_j$ (total flow into node j is greater than or equal to demand d_j). The words "Supplies" and "Demands" are underlined. The context of the course is crucial for understanding the different types of network flow problems and their applications in linear programming. This slide specifically contrasts with the next slide which will likely show the equality constraints version of the transportation problem.

Figures

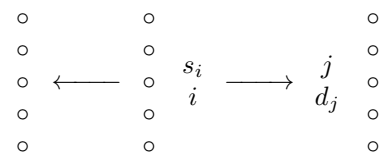


(a) Handwritten notes on variants of network flows, specifically the transportation problem.

Slide 3

Content

Variants of Network Flows
Inequality constraints (Transportation Problem)
Supplies Demands



x_{ij}
x_{id}
x_{dj}

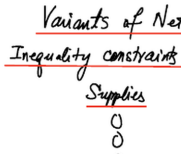
$$\sum_j x_{ij} + x_{id} = s_i, \qquad \sum_i x_{ij} = d_j + x_{dj}$$

$$x_{id}, x_{dj} \geq 0$$

Description

Slide 3 of 21 in a Linear Programming course continues the discussion of network flow problems, specifically addressing the transportation problem with inequality constraints. The title "Variants of Network Flows" is underlined. The slide focuses on how to handle the case where supply exceeds demand (or vice versa) by introducing a "dummy node" (denoted by d). The diagram shows a source node i with supplies s_i and a destination node j with demands d_j. Flows between nodes are represented by x_{ij}, x_{id}, and x_{dj}. x_{ij} represents the flow from source i to destination j, x_{id} represents the flow from source i to the dummy node, and x_{dj} represents the flow from the dummy node to destination j. The key constraints are modified to accommodate the dummy node: $\sum_j x_{ij} + x_{id} = s_i$ (total flow out of node i equals supply s_i) and $\sum_i x_{ij} = d_j + x_{dj}$ (total flow into node j equals demand d_j plus flow from the dummy node). The non-negativity constraint $x_{id}, x_{dj} \geq 0$ is explicitly stated. The words "Supplies" and "Demands" are underlined. The introduction of the dummy node transforms the inequality constraints into equality constraints, making the problem solvable using standard transportation problem algorithms. The context of the course is crucial for understanding how to model and solve transportation problems with unbalanced supply and demand.

Figures



(a) Handwritten notes on variants of network flows, specifically the transportation problem with a dummy node.

Slide 4

Content

Variants of Network Flows
Inequality constraints (Transportation Problem)

$$\sum_j x_{ij} + x_{id} = s_i, \qquad \sum_i x_{ij} = d_j + x_{dj}$$

$$\sum_j x_{ij} + \sum_i x_{id} = \sum_i s_i, \qquad \sum_j x_{ij} = \sum_j d_j + \sum_j x_{dj}$$

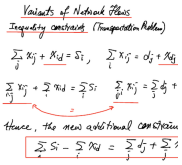
Hence, the new additional constraint:

$$\boxed{\sum_i s_i - \sum_i x_{id} = \sum_j d_j + \sum_j x_{dj}}$$

Description

Slide 4 of 21 in a Linear Programming course continues the discussion of the transportation problem with unbalanced supply and demand, building upon the introduction of a dummy node in the previous slide. The title "Variants of Network Flows" is underlined. The slide presents the constraints from the previous slide: $\sum_j x_{ij} + x_{id} = s_i$ and $\sum_i x_{ij} = d_j + x_{dj}$, where x_{ij} represents the flow from source i to destination j, x_{id} represents the flow from source i to the dummy node, and x_{dj} represents the flow from the dummy node to destination j. These equations are then summed over all i and j respectively, leading to $\sum_j x_{ij} + \sum_i x_{id} = \sum_i s_i$ and $\sum_j x_{ij} = \sum_j d_j + \sum_j x_{dj}$. By equating the left-hand sides of these two equations, a new additional constraint is derived: $\sum_i s_i - \sum_i x_{id} = \sum_j d_j + \sum_j x_{dj}$. This constraint is highlighted within a red box. The words "Inequality constraints" and "Transportation Problem" are underlined. The derivation shows how to create an additional constraint to handle the inequality constraints of the transportation problem by introducing a dummy node and balancing the total supply and demand. The context of the course is crucial for understanding how to manipulate and solve network flow problems with unbalanced supply and demand using linear programming techniques.

Figures



(a) Handwritten notes on variants of network flows, specifically the transportation problem with a dummy node and the derivation of an additional constraint.

Slide 5

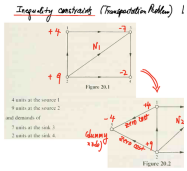
Content

Inequality constraints (Transportation Problem) [C$_7$] p. 321
$[scale = 0.8][circle, draw, label = above : 1](1)at(0, 2); [circle, draw, label = left : 2](2)at(0, 0); [circle, draw, label = above : 3](3)at(3, 2); [circle, draw, label = right : 4](4)at(3, 0); [- >](1) - -node[above]-7(3); [- >](3) - -node[right]-2(4); [- >](4)--node[below](2); [- >](2)--node[left](1); [- >](2)--node[above, sloped](3); at(1.5, 1)N_1$; [red] at (-0.5,2) +4; [red] at (-0.5,0) +9; $\longrightarrow$ $[scale = 0.8][circle, draw, label = above : 1](1)at(0, 2); [circle, draw, label = left : 2](2)at(0, 0); [circle, draw, label = above : 3](3)at(3, 2); [circle, draw, label = right : 4](4)at(3, 0); [circle, draw, label = left : 5](5)at(-1.5, 1); [- >](1)--node[above]-7(3); [- >](3)--node[right]-2(4); [- >](4)--node[below](2); [- >](2) - -node[left](1); [- >](2) - -node[above, sloped](3); [- >](5) - -node[above, sloped]zerocost(1); [- >](5) - -node[below, sloped]zerocost(2); at(1.5, 1)N_2$; [red] at (-0.5,2) +4; [red] at (-0.5,0) +9; [red] at (-2,1) −4; at (-1.5,0.5) (dummy); at (-1.5,0.2) node);
4 units at the source 1
9 units at the source 2
and demands of
7 units at the sink 3
2 units at the sink 4.

Description

Slide 5 of 21 in a Linear Programming course illustrates the transformation of an inequality constrained transportation problem into an equivalent problem with equality constraints using a dummy node. The slide shows two network flow diagrams. Figure 20.1 depicts a transportation problem with unbalanced supply and demand. The nodes represent sources and sinks with associated supplies and demands (4 units at source 1, 9 units at source 2, 7 units demanded at sink 3, and 2 units demanded at sink 4). The arcs represent transportation routes with associated costs (indicated by the numbers on the arcs). Figure 20.2 shows the same problem but with the addition of a dummy node (node 5) and zero-cost arcs connecting the dummy node to the sources and sinks. This addition transforms the inequality constraints into equality constraints, allowing the use of standard transportation problem algorithms. The transformation is explained by the text and the arrows connecting the two figures. The notation [C_7] p. 321 likely refers to a citation or page number in a textbook. The context of the course is crucial for understanding how to model and solve transportation problems with unbalanced supply and demand using linear programming techniques.

Figures



(a) Handwritten notes showing two network flow diagrams, before and after the introduction of a dummy node. The diagrams illustrate the transformation of an inequality constrained transportation problem into an equivalent problem with equality constraints.

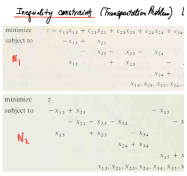
Slide 6

Content

Description

Slide 6 of 21 in a Linear Programming course presents two linear programming formulations of a transportation problem with inequality constraints. The first formulation, labeled N_1, shows a minimization problem with inequality constraints. The objective function is to minimize $z = \sum c_{ij}x_{ij}$, where c_{ij} represents the cost of transporting from source i to destination j, and x_{ij} is the amount transported. The constraints represent the supply and demand limitations. The second formulation, labeled N_2, is an equivalent formulation that transforms the inequality constraints of N_1 into equality constraints by introducing slack variables x_{15} and x_{25}. This transformation is a common technique in linear programming to convert inequality problems into equality problems, which are often easier to solve using standard algorithms. The notation $[C_7]$ p. 321 likely refers to a citation or page number in a textbook. The context of the course is crucial for understanding how to model and solve transportation problems with inequality constraints using linear programming techniques, specifically the method of converting inequalities to equalities by introducing slack variables.

Figures



(a) Handwritten notes showing two linear programming formulations of a transportation problem with inequality constraints. The first formulation (N_1) uses inequality constraints, while the second formulation (N_2) introduces additional variables (x_{15} and x_{25}) to transform the inequalities into equalities.

Slide 7

Content

Inequality constraints (More generally, [?] p.322)

$\begin{bmatrix} A_1 \\ A_2 \end{bmatrix}$ - incident

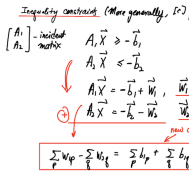
matrix $\begin{matrix} \overrightarrow{A_1}\vec{X} \geq -\overrightarrow{b_1} \\ \overrightarrow{A_2}\vec{X} \leq -\overrightarrow{b_2} \end{matrix} \longrightarrow \begin{matrix} \overrightarrow{A_1}\vec{X} = -\overrightarrow{b_1} + \overrightarrow{W_1}, & \overrightarrow{W_1} \geq \overrightarrow{0} \\ \overrightarrow{A_2}\vec{X} = -\overrightarrow{b_2} - \overrightarrow{W_2}, & \overrightarrow{W_2} \geq \overrightarrow{0} \end{matrix}$

$$\boxed{\sum_p W_{1p} - \sum_q W_{2q} = \sum_p b_{1p} + \sum_q b_{2q}}$$ new constraint

Description

Slide 7 of 21 in a Linear Programming course discusses the handling of inequality constraints in linear programs. The slide begins by presenting a system of inequality constraints, $\overrightarrow{A_1}\vec{X} \geq -\overrightarrow{b_1}$ and $\overrightarrow{A_2}\vec{X} \leq -\overrightarrow{b_2}$, where $\overrightarrow{A_1}$ and $\overrightarrow{A_2}$ are matrices, $\vec{X}$ is a vector of variables, and $\overrightarrow{b_1}$ and $\overrightarrow{b_2}$ are vectors of constants. The matrices A_1 and A_2 are described as forming an "incident matrix," suggesting a context related to network flows or graph theory. The inequalities are then transformed into equalities by introducing slack variables $\overrightarrow{W_1}$ and $\overrightarrow{W_2}$: $\overrightarrow{A_1}\vec{X} = -\overrightarrow{b_1} + \overrightarrow{W_1}$ and $\overrightarrow{A_2}\vec{X} = -\overrightarrow{b_2} - \overrightarrow{W_2}$, with non-negativity constraints $\overrightarrow{W_1} \geq \overrightarrow{0}$ and $\overrightarrow{W_2} \geq \overrightarrow{0}$. Finally, a new constraint is derived by summing the transformed equations, resulting in $\sum_p W_{1p} - \sum_q W_{2q} = \sum_p b_{1p} + \sum_q b_{2q}$. This new constraint is highlighted in a red box, indicating its importance in solving the linear program. The notation "[?]" p.322 likely refers to a citation or page number in a textbook. The context of the course is crucial for understanding how to manipulate and solve linear programs with inequality constraints by converting them into equality constraints using slack variables and deriving additional constraints.

Figures



(a) Handwritten notes showing the transformation of inequality constraints in a linear program into equality constraints by introducing slack variables. The notes include a matrix notation for the constraints, the transformation process, and a new constraint derived from the transformation.

Slide 8

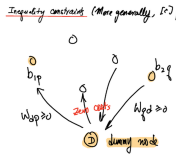
Content

Inequality constraints (More generally, [?] p.322)
[scale=0.8] [circle,draw,label=above:b_{1p}] (1) at (0,2) ; [circle,draw] (2) at (1.5,1) ; [circle,draw,label=above:b_{2q}] (3) at (3,2) ; [circle,draw,fill=orange!50,label=below:D] (4) at (1.5,0) ; [->] (4) – node[left] $W_{dp} \geq 0$ (1); [->] (2) – node[left] (4); [->] (3) – node[right] $W_{dq} \geq 0$ (4); [red] at (1.5,0.7) zero costs; at (1.5,-0.5) dummy node;

Description

Slide 8 of 21 in a Linear Programming course illustrates a general method for handling inequality constraints in linear programs, particularly in the context of network flow problems. The slide shows a network flow diagram with a central "dummy node" (D). This dummy node is connected to other nodes representing sources (b_{1p}) and sinks (b_{2q}) with arcs representing flows (W_{dp} and W_{dq}). The arcs connecting to the dummy node have zero costs ("zero costs" is explicitly written). The variables W_{dp} and W_{dq} represent slack variables, and the non-negativity constraints ($W_{dp} \geq 0$ and $W_{dq} \geq 0$) are explicitly shown. The diagram visually represents the transformation of inequality constraints into equality constraints by introducing a dummy node and associated slack variables. The notation "[?]" p.322 likely refers to a citation or page number in a textbook. The context of the course is crucial for understanding how to model and solve linear programs with inequality constraints using network flow techniques and the introduction of dummy nodes to handle imbalances in supply and demand.

Figures



(a) Handwritten notes showing a network flow diagram with a dummy node. The diagram illustrates a more general approach to handling inequality constraints in linear programming by introducing a dummy node with zero-cost arcs.

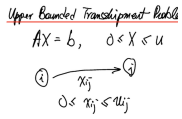
Slide 9

Content

Upper Bounded Transshipment Problem [V] p.263
AX = b, $0 \leq X \leq U$[scale=0.8] [circle,draw] (i) at (0,0) i; [circle,draw] (j) at (3,0) j; [->] (i) to[bend left] node[below] x_{ij}

Slide 9 of 21 in a Linear Programming course introduces the "Upper Bounded Transshipment Problem." The slide presents the problem's formulation: $AX = b$, where A is a matrix, X is a vector of variables representing flows, and b is a vector of constants representing supply and demand. Crucially, the variables X are bounded above by a vector U, represented as $0 \leq X \leq U$. This indicates that each flow x_{ij} has an upper limit u_{ij}. A small diagram visually depicts a flow x_{ij} from node i to node j, with the constraint $0 \leq x_{ij} \leq u_{ij}$ written below. The notation "[V]" likely refers to a specific problem variant or section in a textbook, and "p.263" indicates a page number. The context of the course is essential for understanding the specific meaning of the matrix A and vectors X, b, and U within the context of network flow problems and linear programming.

Figures



(a) Handwritten notes showing the formulation of an upper bounded transshipment problem. The notes include a matrix equation $AX = b$ with bounds on the variables $0 \leq X \leq U$, and a small diagram illustrating the flow x_{ij} between nodes i and j with upper bound u_{ij}.

Slide 10

Content

Upper Bounded Transshipment Problem
AX = -b, $\quad 0 \leq X \leq U$
$[scale = 0.8][circle, draw](i)at(0,0)$i; $[circle, draw](j)at(3,0)$j; $[->](i)to[bendleft]node[above]x_{ij}$ (j); [->] (i) to[out=150,in=180,looseness=1] (i); [->] (i) to[out=135,in=180,looseness=1] (i); [->] (i) to[out=120,in=180,looseness=1] (i); [->] (j) to[out=30,in=0,looseness=1] (j); [->] (j) to[out=15,in=0,looseness=1] (j); [->] (j) to[out=-15,in=0,looseness=1] (j); at (0,0.5) b_i; at (3,0.5) b_j; at (1.5,-0.5) $0 \leq x_{ij} \leq u_{ij}$;

$\cdots \quad -x_{ij} \quad \cdots = -b_i$

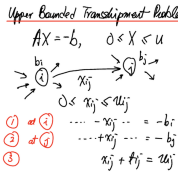
$\cdots \quad +x_{ij} \quad \cdots = -b_j$

x$_{ij} + t_{ij} = u_{ij} \quad (t_{ij} \geq 0)$

Description

Slide 10 of 21 in a Linear Programming course details the Upper Bounded Transshipment Problem. The slide begins by stating the problem's formulation: $AX = -b, 0 \leq X \leq U$, where A is the incidence matrix, X is the vector of flows, b is the vector of supply/demand, and U is the vector of upper bounds on the flows. A small network flow diagram illustrates the flow x_{ij} between nodes i and j, subject to the constraint $0 \leq x_{ij} \leq u_{ij}$. Multiple arrows entering and leaving nodes i and j represent multiple sources and destinations connected to each node. Below the diagram, the slide shows a system of equations representing flow conservation at nodes i and j. The equation $x_{ij} + t_{ij} = u_{ij}$ introduces a slack variable $t_{ij} \geq 0$ to handle the upper bound constraint. The context of the course makes it clear that this is a network flow problem where the goal is to find the optimal flow satisfying supply/demand constraints and upper bounds on arc capacities. The negative sign in $AX = -b$ is a convention often used in network flow formulations.

Figures



(a) Handwritten notes showing the formulation of an upper bounded transshipment problem. The notes include a matrix equation $AX = -b$ with bounds on the variables $0 \leq X \leq U$, a small diagram illustrating the flow x_{ij} between nodes i and j with upper bound u_{ij}, and a system of equations representing the flow balance constraints at nodes i and j. The diagram also shows multiple arrows entering and leaving nodes i and j, indicating multiple sources and destinations.

Slide 11

Content

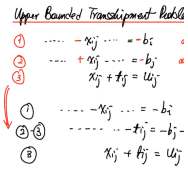
Upper Bounded Transshipment Problem

$$
\begin{array}{lll}
1 & \cdots - x_{ij} \cdots & = -b_i \quad \text{at } i \\
2 & \cdots + x_{ij} \cdots & = -b_j \quad \text{at } j \\
 & x_{ij} + t_{ij} & = u_{ij} \\
1 & \cdots - x_{ij} \cdots & = -b_i \\
2 & \cdots\cdots\cdots - t_{ij} & = -b_j - u_{ij} \\
 & x_{ij} + t_{ij} & = u_{ij}
\end{array}
$$

Description

Slide 11 of 21 in a Linear Programming course presents a method for solving the Upper Bounded Transshipment Problem. The slide shows two systems of equations. The first system represents flow conservation at nodes i and j in a network, where x_{ij} is the flow from node i to node j, b_i and b_j are the net supply/demand at nodes i and j, and u_{ij} is the upper bound on the flow x_{ij}. The equation $x_{ij} + t_{ij} = u_{ij}$ introduces a slack variable t_{ij} to handle the upper bound constraint. The second system is a transformation of the first, where the upper bound constraint is incorporated into the flow balance equation at node j. The red arrow and circled numbers suggest a step-by-step transformation of the equations. The context of the course is crucial for understanding the meaning of the variables and the transformation of the equations to solve the problem efficiently. The slide demonstrates a technique for handling upper bounds in network flow problems within the broader context of linear programming.

Figures



(a) Handwritten notes showing the formulation of an upper bounded transshipment problem. The notes include a system of equations representing the flow balance constraints at nodes i and j, and a transformation of the equations to incorporate slack variables. The equations are numbered (1), (2), and (3) and the nodes are also circled (1), (2), and (3). A red arrow points from the first set of equations to the second set, indicating a transformation.

Slide 12

Content

Upper Bounded Transshipment Problem

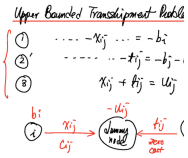
$$
\begin{aligned}
1 \quad \cdots - x_{ij} \cdots \quad &= -b_i \\
2 \quad \cdots\cdots\cdots - t_{ij} \quad &= -b_j - u_{ij} \\
3 \quad x_{ij} + t_{ij} \quad &= u_{ij}
\end{aligned}
$$

[scale=0.8] [circle,draw] (i) at (0,0) i; [circle,draw] (dummy) at (3,0) dummy
node; [circle,draw] (j) at (6,0) j; [->] (i) to node[above] x_{ij} node[below] c_{ij} (dummy); [<-] (j) to node[above] t_{ij} node[below] zero
cost (dummy); at (0,0.5) b_i; at (3,0.5) $-u_{ij}$; at (6,0.5) $b_j + u_{ij}$;

Description

Slide 12 of 21 in a Linear Programming course demonstrates a technique for solving the Upper Bounded Transshipment Problem using a dummy node. The slide presents a system of three equations. The first two equations represent flow conservation at nodes i and j, modified to incorporate the upper bound u_{ij} on the flow x_{ij} from i to j. The third equation defines a slack variable t_{ij} representing the unused capacity on the arc from i to j. The diagram illustrates the addition of a "dummy node" to the network. The flow x_{ij} goes from node i to the dummy node, and the flow t_{ij} goes from the dummy node to node j. The dummy node has a supply of $-u_{ij}$, effectively representing the unused capacity. The cost of the arc from the dummy node to node j is zero. This transformation converts the upper-bounded problem into a standard transshipment problem that can be solved using simpler algorithms. The circled numbers (1), (2), and (3) indicate the order of the equations. The context of the course is crucial for understanding the transformation and the role of the dummy node in handling upper bounds.

Figures



(a) Handwritten notes showing a system of three equations representing flow balance constraints in a network with an added dummy node to handle upper bounds. A diagram illustrates the flow x_{ij} from node i to a dummy node, and a flow t_{ij} from the dummy node to node j. The nodes are labeled with their net supply/demand values, and the arcs are labeled with flow variables and costs. The equations are numbered (1), (2), and (3) and the nodes are also circled (1), (2), and (3).

Slide 13

Content

Shortest Path Problem [scale=0.8] [circle,draw] (s) at (0,0) S; [circle,draw] (a) at (2,1) ; [circle,draw] (b) at (2,-1) ; [circle,draw] (c) at (4,0) ; [circle,draw] (d) at (6,1) ; [circle,draw] (e) at (6,-1) ; [circle,draw] (r) at (8,0) R; [->,red!50!white,line width=2pt] (s) to (a); [->,red!50!white,line width=2pt] (a) to (c); [->,red!50!white,line width=2pt] (c) to (d); [->,red!50!white,line width=2pt] (d) to (r); [->] (a) to (b); [->] (b) to (c); [->] (c) to (e); [->] (e) to (r); [->] (a) to (c); at (4,0.3) i; at (6,1.3) j;

Description

Slide 13 of 21 in a Linear Programming course illustrates a shortest path problem. The slide features a handwritten diagram of a directed graph. The graph represents a network with nodes labeled S (source) and R (destination), and intermediate nodes labeled i and j. The edges are represented by arrows indicating direction. A path from S to R is highlighted in salmon/light red, suggesting this is the shortest path found by some algorithm (not shown). The context of the course makes it clear that this is a visual representation of a problem that can be formulated and solved using linear programming techniques. The diagram is a simple example, likely used to introduce the concept before moving on to more complex network flow problems.

Figures

(a) A handwritten diagram showing a graph representing a shortest path problem. The graph has nodes labeled S (source) and R (destination), and intermediate nodes labeled i and j. The shortest path is highlighted in salmon/light red. Arrows indicate the direction of the edges. The nodes are circles, and the edges are lines with arrows.

Slide 14

Content

Shortest Path Problem
[scale=0.8] [circle,draw] (s) at (0,0) S; [circle,draw] (a) at (1.5,0.5) ; [circle,draw] (b) at (1.5,-0.5) ; [circle,draw] (c) at (3,0.5) ; [circle,draw] (d) at (3,-0.5) ; [circle,draw] (e) at (4.5,0.5) ; [circle,draw] (f) at (4.5,-0.5) ; [circle,draw] (r) at (6,0) R; [->,red!50!white,line width=2pt] (s) to (a); [->,red!50!white,line width=2pt] (a) to (c); [->,red!50!white,line width=2pt] (c) to (e); [->,red!50!white,line width=2pt] (e) to (r); [->] (a) to (b); [->] (b) to (d); [->] (d) to (f); [->] (f) to (r); [->] (a) to (c); [->] (c) to (e); at (0,-0.3) $+1$; at (6,-0.3) -1; at (3,1) c_{ij}; at (3.75,1) $b_j = 0$; at (3,0) i; at (4.5,0) j;
Network flow formulation

$$\min \sum_{ij} c_{ij} x_{ij}$$

$$\text{s.t.} \sum_{j} x_{sj} = 1, \quad \sum_{i} x_{ir} = 1$$

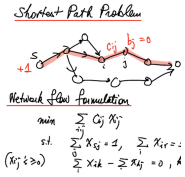
$$\sum_{i} x_{ik} - \sum_{j} x_{kj} = 0, \quad k \neq s, r$$

$$(x_{ij} \geq 0)$$

Description

Slide 14 of 21 in a Linear Programming course presents a shortest path problem formulated as a network flow problem. The diagram shows a directed graph with nodes S (source) and R (destination), and intermediate nodes. A shortest path from S to R is highlighted in salmon/light red. The source node S has a supply of +1, and the destination node R has a demand of -1, indicating that one unit of flow must traverse from S to R. The network flow formulation is given below the diagram. The objective function is to minimize $\sum_{ij} c_{ij} x_{ij}$, where c_{ij} is the cost of traversing edge (i, j) and x_{ij} is the flow on that edge. The constraints ensure flow conservation at each node: $\sum_{j} x_{sj} = 1$ means one unit of flow leaves the source S; $\sum_{i} x_{ir} = 1$ means one unit of flow arrives at the destination R; and $\sum_{i} x_{ik} - \sum_{j} x_{kj} = 0$ for all nodes k other than S and R, ensures that flow entering a node equals flow leaving the node. The constraint $x_{ij} \geq 0$ indicates that flow is non-negative. The context of the course makes it clear that this is a linear programming formulation of a shortest path problem, which can be solved using linear programming techniques.

Figures



(a) A handwritten diagram showing a graph representing a shortest path problem. The graph has nodes labeled S (source) and R (destination), and intermediate nodes labeled i and j. The shortest path is highlighted in salmon/light red. Arrows indicate the direction of the edges. The nodes are circles, and the edges are lines with arrows. The source node S has a value of +1 and the destination node R has a value of -1. The network flow formulation is written below the diagram, including an objective function to minimize the total cost and constraints representing flow conservation at each node. The constraints ensure that one unit of flow leaves the source and one unit arrives at the destination.

Slide 15

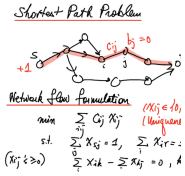
Content

Shortest Path Problem
[scale=0.8] [circle,draw] (s) at (0,0) S; [circle,draw] (a) at (1.5,0.5) ; [circle,draw] (b) at (1.5,-0.5) ; [circle,draw] (c) at (3,0) ; [circle,draw] (d) at (4.5,0.5) ; [circle,draw] (e) at (4.5,-0.5) ; [circle,draw] (r) at (6,0) R; [->,red!50!white,line width=2pt] (s) to (a); [->,red!50!white,line width=2pt] (a) to (c); [->,red!50!white,line width=2pt] (c) to (d); [->,red!50!white,line width=2pt] (d) to (r); [->] (a) to (b); [->] (b) to (c); [->] (c) to (e); [->] (e) to (r); at (0,-0.3) $+1$; at (6,-0.3) -1; at (3,0.3) i; at (4.5,0.3) j; at (3.75,0.8) c_{ij}; at (3.75,1.2) $b_j = 0$;
Network flow formulation
$\min \sum_{ij} c_{ij} x_{ij}$
s.t. $\sum_j x_{sj} = 1, \quad \sum_i x_{ir} = 1$
$\sum_i x_{ik} - \sum_j x_{kj} = 0, \quad k \neq s, r$
$(x_{ij} \geq 0)$
$(x_{ij} \in \{0, 1\}?)$
(Uniqueness?)

Description

Slide 15 of 21 in a Linear Programming course shows how to formulate a shortest path problem as a network flow problem. The diagram depicts a directed graph with nodes S (source, supply +1) and R (destination, demand -1), and intermediate nodes i and j. A shortest path from S to R is highlighted in light red. The network flow formulation is presented below: the objective function minimizes the total cost $\sum_{ij} c_{ij} x_{ij}$, subject to flow conservation constraints. The constraint $\sum_j x_{sj} = 1$ ensures one unit of flow leaves the source, and $\sum_i x_{ir} = 1$ ensures one unit arrives at the destination. The constraint $\sum_i x_{ik} - \sum_j x_{kj} = 0$ (for $k \neq s, r$) enforces flow conservation at intermediate nodes. The non-negativity constraint $x_{ij} \geq 0$ is included. Importantly, a question mark is added next to the constraint $x_{ij} \in \{0, 1\}?$, indicating a discussion point about whether the flow variables should be restricted to binary values (0 or 1), which is typical for shortest path problems. The question "(Uniqueness?)" at the end suggests a discussion about the uniqueness of the shortest path solution. The context of the course (Linear Programming) is crucial because it shows how a combinatorial optimization problem (shortest path) can be formulated and solved using linear programming techniques.

Figures



(a) A handwritten diagram showing a graph representing a shortest path problem. The graph has nodes labeled S (source) and R (destination), and intermediate nodes labeled i and j. The shortest path is highlighted in salmon/light red. Arrows indicate the direction of the edges. The nodes are circles, and the edges are lines with arrows. The source node S has a value of +1 and the destination node R has a value of -1. The network flow formulation is written below the diagram, including an objective function to minimize the total cost and constraints representing flow conservation at each node. The constraints ensure that one unit of flow leaves the source and one unit arrives at the destination. There is a question mark next to the constraint $x_{ij} \in \{0, 1\}$?, indicating uncertainty about whether the flow variables should be restricted to binary values. The question "(Uniqueness?)" suggests a concern about whether the shortest path is unique.

Slide 16

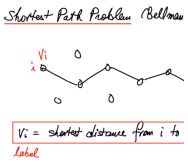
Content

Shortest Path Problem (Bellman's Eqn)
[scale=0.8] [circle,draw] (i) at (0,0) ; [circle,draw] (a) at (1.5,-0.5) ; [circle,draw] (b) at (3,0) ; [circle,draw] (c) at (4.5,-0.5) ; [circle,draw,fill=red!20] (r) at (6,0) ; (i) to (a); (a) to (b); (b) to (c); (c) to (r); [left] at (i) i; [above] at (i) V_i; [right] at (r) r; [above] at (1.5,0) 0; [below] at (1.5,-0.5) 0; [above] at (3,0.5) 0; [below] at (4.5,-0.5) 0;
V_i = shortest distance from i to r
label

Description

Slide 16 of 21 in a Linear Programming course illustrates the shortest path problem using Bellman's equation. The slide features a handwritten diagram of a directed acyclic graph. The graph represents a network with nodes labeled i (source) and r (destination), and intermediate nodes. The edges are represented by lines with arrows indicating direction. All edge weights are 0. The destination node r is highlighted with a light pink/salmon fill. A box defines V_i as the shortest distance from node i to node r. The context of the course makes it clear that this is a visual representation of a problem that can be solved using Bellman's equation, a dynamic programming approach often used to solve shortest path problems. The simplicity of the graph (all weights are 0) suggests this is an introductory example before moving to more complex scenarios.

Figures



(a) A handwritten diagram showing a graph representing a shortest path problem. The graph has nodes labeled i (source) and r (destination), and intermediate nodes. The destination node r is lightly shaded in pink/salmon. Arrows indicate the direction of the edges. The nodes are circles, and the edges are lines. The weights on the edges are all 0. A box contains the definition of V_i as the shortest distance from node i to node r.

Slide 17

Content

Shortest Path Problem (Bellman's Eqn)
[scale=0.8] [circle,draw] (i) at (0,0) i; [circle,draw] (j) at (1.5,0) j; [circle,draw] (k) at (2.5,0.2) ; [circle,draw] (l) at (3.5,0.1) ; [circle,draw] (r) at (4.5,0) r; [->] (i) to node[above] c_{ij} (j); (j) to (k); (k) to (l); (l) to (r); [left] at (i) V_i; [right] at (j) V_j;

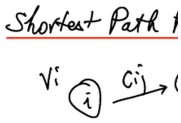
$$\boxed{V_i = \min_j \{c_{ij} + V_j\}}$$

$V_r = 0$

Description

Slide 17 of 21 in a Linear Programming course introduces Bellman's equation for solving the shortest path problem. A simple directed acyclic graph is shown, with nodes i (source), j, k, l, and r (destination). The edge from node i to node j has a weight c_{ij}. The nodes i and j are labeled with V_i and V_j respectively, representing the shortest distance from that node to the destination node r. Bellman's equation, $V_i = \min_j \{c_{ij} + V_j\}$, is presented in a box. This equation recursively defines the shortest distance from node i to the destination node r as the minimum of the sum of the weight of the edge from i to j and the shortest distance from j to r, considering all possible neighboring nodes j. The equation $V_r = 0$ indicates that the shortest distance from the destination node r to itself is 0. The context of the course (Linear Programming) is important because dynamic programming techniques, such as Bellman's equation, are used to solve optimization problems, including shortest path problems, which can be formulated as linear programs.

Figures



(a) A handwritten diagram showing a graph representing a shortest path problem. The graph has nodes labeled i (source) and r (destination), and intermediate nodes j, k, l. Arrows indicate the direction of the edges. The nodes are circles, and the edges are lines with arrows. The weight on the edge from i to j is labeled c_{ij}. The shortest distance from node i to node r is defined by Bellman's equation, which is written in a box below the diagram. The destination node r is not explicitly labeled as the destination, but it is implied by the context and the equation.

Slide 18

Content

Shortest Path Problem (Bellman's Eqn)

$$\boxed{V_i = \min_j \{c_{ij} + V_j\}, \quad V_r = 0}$$

Approximation/Iteration

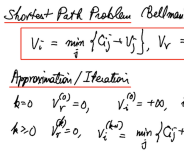
k=0 $\quad V_r^{(0)} = 0, \quad V_i^{(0)} = +\infty, \quad i \neq r$

$k > 0 \quad V_r^{(k)} = 0, \quad V_i^{(k+1)} = \min_j \{c_{ij} + V_j^{(k)}\}$

Description

Slide 18 of 21 in a Linear Programming course presents Bellman's equation for solving the shortest path problem and an iterative method for approximating its solution. The core equation, $V_i = \min_j \{c_{ij} + V_j\}$, is boxed, where V_i represents the shortest distance from node i to a destination node r, c_{ij} is the cost of the edge from node i to node j, and the minimization is taken over all neighboring nodes j. The condition $V_r = 0$ sets the shortest distance from the destination node to itself to zero. Below, an iterative approximation scheme is described. The algorithm initializes $V_r^{(0)} = 0$ (shortest distance from r to r) and $V_i^{(0)} = +\infty$ for all other nodes i. Then, it iteratively updates the values $V_i^{(k)}$ using the Bellman equation, where the superscript (k) denotes the iteration number. The iterative process continues until convergence, providing an approximation of the shortest path distances. The context of the course (Linear Programming) is important because this dynamic programming approach is a common method for solving shortest path problems, which can be formulated as linear programs.

Figures



(a) Handwritten notes showing Bellman's equation for the shortest path problem and an iterative approximation method. The equation $V_i = \min_j \{c_{ij} + V_j\}$ is boxed, with $V_r = 0$. Below, an iterative approach is described, starting with $V_r^{(0)} = 0$ and $V_i^{(0)} = +\infty$ for $i \neq r$, and then recursively updating $V_i^{(k+1)}$ using the boxed equation.

Slide 19

Content

Shortest Path Problem (Bellman's Eqn)

$$\boxed{V_i = \min_j \{c_{ij} + V_j\}, \quad V_r = 0}$$

Approximation/Iteration

k=0 $\quad V_r^{(0)} = 0, \quad V_i^{(0)} = +\infty, \quad i \neq r$

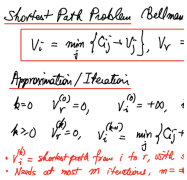
$k > 0 \quad V_r^{(k)} = 0, \quad V_i^{(k+1)} = \min_j \{c_{ij} + V_j^{(k)}\}$

- $V_i^{(k)}$ = shortest path from i to r with $\leq k$ arcs
- Needs at most m iterations, $m = \#$ of nodes

Description

Slide 19 of 21 in a Linear Programming course details an iterative approximation method for solving the shortest path problem using Bellman's equation. The slide begins by presenting the Bellman equation in a box: $V_i = \min_j \{c_{ij} + V_j\}$, where V_i is the shortest distance from node i to the destination node r, and c_{ij} is the cost of the edge from node i to node j. The condition $V_r = 0$ is also given. An iterative approximation scheme is then described. The algorithm initializes $V_r^{(0)} = 0$ and $V_i^{(0)} = +\infty$ for all $i \neq r$. It iteratively updates $V_i^{(k)}$ using the Bellman equation, where the superscript (k) denotes the iteration number. Two bullet points clarify that $V_i^{(k)}$ represents the shortest path from node i to the destination node r using at most k arcs, and that the algorithm converges in at most m iterations, where m is the number of nodes in the graph. The context of linear programming is crucial because this dynamic programming approach is a common method for solving shortest path problems, which can be formulated as linear programs. The handwritten notes suggest a more informal and explanatory style compared to the previous slides.

Figures



(a) Handwritten notes showing Bellman's equation for the shortest path problem and an iterative approximation method. The equation $V_i = \min_j \{c_{ij} + V_j\}$ is boxed, with $V_r = 0$. Below, an iterative approach is described, starting with $V_r^{(0)} = 0$ and $V_i^{(0)} = +\infty$ for $i \neq r$, and then recursively updating $V_i^{(k+1)}$ using the boxed equation. Two bullet points describe the meaning of $V_i^{(k)}$ and the number of iterations needed for convergence.

Slide 20

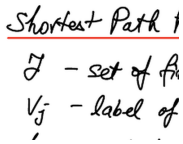
Content

Shortest Path Problem (Dijkstra Alg.)
J - set of finished node
V_j – label of j (shortest distance j to r)
h_j – node to visit next in shortest path

Description

Slide 20 of 21 in a Linear Programming course describes the Dijkstra algorithm for finding the shortest path in a graph. The slide defines three key variables used in the algorithm: $\mathcal{J}$ represents the set of nodes whose shortest distance to the destination node r has already been determined (the "finished" nodes). V_j represents the label of node j, which is the length of the shortest path found so far from node j to the destination node r. Finally, h_j represents the next node to be visited along the shortest path from node j to node r. The context of linear programming is relevant because shortest path problems can be formulated and solved using linear programming techniques. The Dijkstra algorithm provides an efficient alternative to these linear programming methods for solving shortest path problems.

Figures



(a) Handwritten notes describing the Dijkstra algorithm for finding the shortest path. The notes define $\mathcal{J}$ as the set of finished nodes, V_j as the label of node j (representing the shortest distance from node j to the destination node r), and h_j as the next node to visit in the shortest path from node j to node r.

Slide 21

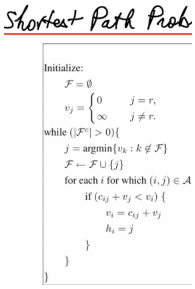
Content

Shortest Path Problem (Dijkstra Alg.)
Initialize:
F = $\emptyset$
$v_j = \begin{cases} 0 & j = r, \\ \infty & j \neq r. \end{cases}$
while $(|\mathcal{F}| > 0)\{$
$j = \text{argmin}\{v_k : k \notin \mathcal{F}\}$
$\mathcal{F} = \mathcal{F} \cup \{j\}$
for each i for which $(i, j) \in \mathcal{A}$ and $i \notin \mathcal{F}\{$
if $(c_{ij} + v_j < v_i)\{$
$v_i = c_{ij} + v_j$
$h_i = j$
}
}
}

Description

Slide 21 of 21 in a Linear Programming course presents the Dijkstra algorithm for solving the shortest path problem. The algorithm is presented in pseudocode. The algorithm initializes an empty set $\mathcal{F}$ representing the set of visited nodes, and a vector v_j representing the shortest distance from node j to the destination node r. v_r is initialized to 0, and all other v_j are initialized to ∞. The main loop iterates while the cardinality of $\mathcal{F}$ is greater than 0. In each iteration, the algorithm selects the node j with the minimum v_j value among unvisited nodes ($j \notin \mathcal{F}$), adds j to $\mathcal{F}$, and updates the distances v_i for all neighbors i of j that are not in $\mathcal{F}$. If a shorter path to i is found through j, v_i is updated, and h_i (the predecessor node in the shortest path to i) is set to j. The algorithm terminates when all nodes have been visited. The notation $\mathcal{A}$ likely represents the adjacency matrix or a similar structure representing the edges in the graph. The context of linear programming is relevant because shortest path problems can be formulated and solved using linear programming techniques, but Dijkstra's algorithm provides a more efficient solution in many cases.

Figures



(a) Handwritten notes showing the Dijkstra algorithm for finding the shortest path. The algorithm initializes a set $\mathcal{F}$ to the empty set and assigns values to v_j based on whether j is the destination node r. The main loop continues as long as the cardinality of $\mathcal{F}$ is greater than 0. Inside the loop, the algorithm finds the node j with the minimum value of v_k among nodes not in $\mathcal{F}$, adds j to $\mathcal{F}$, and updates the values of v_i and h_i for neighboring nodes i not in $\mathcal{F}$ if a shorter path is found.